SYSTEM DESIGN

Connection Pooling Visual Guide

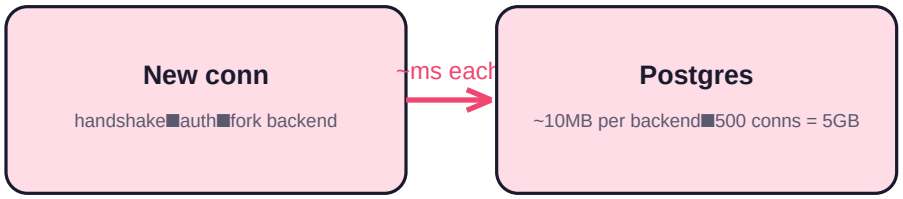
Databases hate connection churn:
reuse, size, and multiplex connections.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Connections Are Expensive

Each one costs a TCP + TLS handshake, auth, and megabytes of server memory.

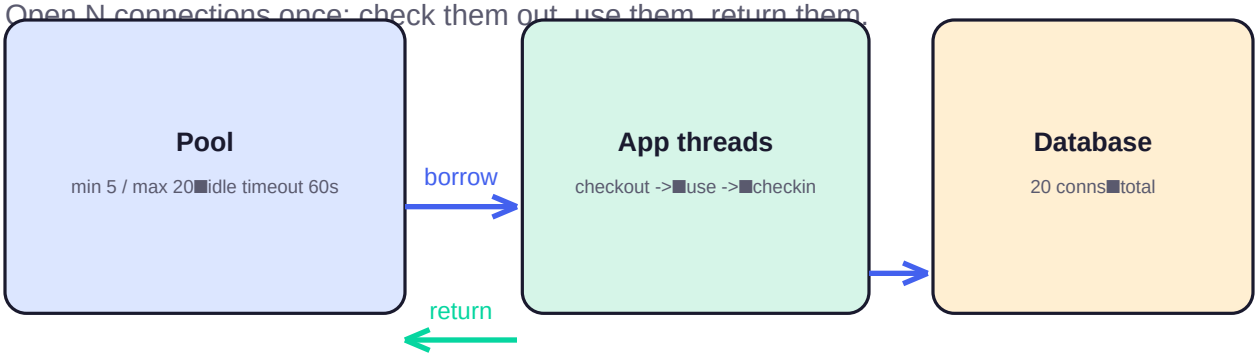


- At 1000 requests/sec, opening a connection per request = 1000 handshakes/sec of pure overhead.
- Server-side, each backend is a PROCESS (Postgres): context switches and memory explode past a few hundred.
- Client-side, TIME_WAIT sockets and port exhaustion bite first -- the pool fixes both ends.

STEP 2/6

The Pool

Open N connections once; check them out, use them, return them.



- Checkout with a TIMEOUT: waiting forever for a connection is a hung request with extra steps.
- Validate on checkout (cheap ping) or pay with stale-connection errors after DB restarts.
- The pool is per PROCESS: 50 app replicas x 20 = 1000 DB connections. Do the multiplication!

STEP 3/6

Pool Sizing Math

Little's Law: connections = throughput x query latency.

```
pool_size = (queries/sec) x (avg seconds/query)

example:  500 qps x 0.02 s  =  10 connections
```

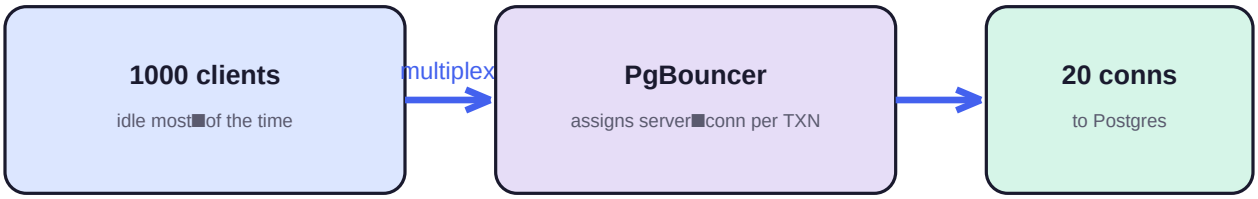
- Measure YOUR latency: 500 qps at 20ms needs 10 conns; at 200ms it needs 100.
- Start small and grow: an idle connection still costs server memory.
- Postgres sweet spot is often cores x 2 + spare -- beyond that, throughput DROPS (contention).

Bigger is not better: past the knee, extra connections just queue inside the DB and slow everyone.

STEP 4/6

PgBouncer: Transaction Pooling

The proxy that multiplexes thousands of clients onto tens of server connections.

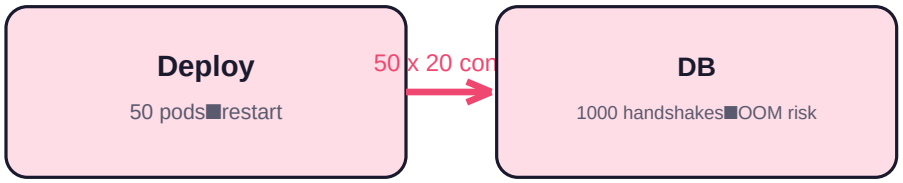


- Transaction mode: a server connection is borrowed per TRANSACTION, not per client -- idle clients hold nothing.
- Session mode (default): one server conn per client -- safe for LISTEN/NOTIFY, temp tables, prepared statements.
- Statement mode: per query -- fastest multiplexing, breaks multi-statement transactions.

STEP 5/6

The Connection Storm

Every app replica restarts at once; the DB drowns in handshakes.



- Defenses: a proxy tier (PgBouncer/RDS Proxy) absorbs the storm; rolling restarts spread it out.
- Serverless trap: 1000 Lambda invocations x cold pool = 1000 connections -- proxy or perish.
- Set max_connections on the DB as a CIRCUIT BREAKER, not a target.

STEP 6/6

Timeouts and Health

A pool without timeouts is a distributed deadlock kit.

- checkout_timeout (2s): fail fast when the pool is exhausted -- surface the queue, do not hide it.
- idle_timeout / max_lifetime (5-30 min): recycle connections before the DB, LB, or NAT kills them silently.
- Health check on checkout: SELECT 1 is cheap; a stale half-open connection is expensive.
- Metrics that matter: pool wait time, checkout failures, idle vs active -- alert on wait time first.

STEP ✓/✓

Key Takeaways

Connection Pooling — Remember

- Connections cost handshakes, auth, and ~10MB server-side each -- never open per request.
- Pools lend connections out and take them back; size with Little's Law (qps x latency).
- Pool count multiplies by app replicas -- a proxy tier (PgBouncer) multiplexes thousands to tens.
- Transaction pooling borrows per transaction; session mode keeps per-client state safe.
- Timeouts on checkout, lifetime caps, and wait-time metrics keep the pool honest.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev